Technical University of Denmark

Written examination date: May 27, 2024



Course title: Computer Science Modelling

Course number: 02141

Aids allowed: All aid

Exam duration: 4 hours

Weighting: 7-step scale

RULES & RECOMMENDATIONS:

**R1** You can submit your answers digitally or as a paper submission, or both. If you submit some answers on paper and some digitally, you must clearly indicate in your digital version (at the respective position for the answer) that the solution is provided on paper and on which page.

**R2** Digital submissions must be done as a single PDF file.

**R3** Some solutions require that you provide code that can be used with the tools seen in the course (fm4fun, inspectify, chip, moka). In those cases, your digital submission must allow copying the code from the PDF such that it can be pasted into those tools. In particular, screenshots are *not* allowed.

**R4** You must clearly indicate on each page your student number, and which (sub)task you attempt to solve.

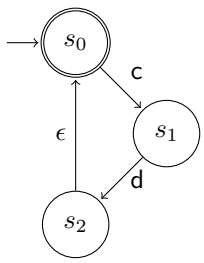
**R5** We recommend that you first skim all tasks before deciding in which order you want to solve them.

**Answers Included**

# Task 1

Barbara wants to build a coffee machine that offers coffee in exchange of donations. She consults with her friends Alicia, Eva, Felipe and Giorgio. They all agree on using d to represent a donation and c to represent a coffee. However, they use different formalisms to represent the model of the coffee machine!

Alicia proposes automaton A:




Eva proposes the regular expression E:

$$(\mathsf{cd})^*$$

Felipe proposes the LTL formula F:

$$\mathsf{G}(\mathsf{c} \to \mathsf{Fd})$$

Giorgio proposes the grammar G:

$$S \to \mathsf{cd} \mid \mathsf{c}S\mathsf{d} \mid SS$$

Note that Felipe is using LTL over **finite** traces, which is a variant of LTL as seen in class. Formally, a finite word $w = a_0 a_1 \dots a_n$ is accepted by an LTL formula $\phi$ iff $w, 0 \models \phi$, where the relation $\models$ is formalised as follows:

$$\begin{array}{lcll}
w, i & \models & a & \text{if } w[i] = a \\
w, i & \models & \phi_1 \to \phi_2 & \text{if } w, i \models \phi_1 \text{ implies } w, i \models \phi_2 \\
w, i & \models & G\phi & \text{if } \forall j \in \{i, .., |w| - 1\}. w, j \models \phi \\
w, i & \models & F\phi & \text{if } \exists j \in \{i, .., |w| - 1\}. w, j \models \phi
\end{array}$$

NOTE: Cases of LTL formulas not covered by F are not included for simplicity.

The language of an LTL formula $\phi$ over an alphabet $\Sigma$ is then defined as $\mathcal{L}(\phi) = \{w \in \Sigma^* \mid w, 0 \models \phi\}$.

All LTL formulas over finite traces can be translated into equivalent DFAs.

**Question 1a:** *Barbara wants to compare the proposed coffee machine models. In particular, she wants to know, for each of model, whether they allow for behaviours that the other models do not admit.*

*To help Barbara,* ***fill the table below****. In the cell corresponding to the row language $L_1$ and to the column language $L_2$ provide a minimal length word that belongs to $L_1 \setminus L_2$. Write "————" if no such word exists (no justification is needed).*

| | $\mathcal{L}(\mathsf{A})$ | $\mathcal{L}(\mathsf{E})$ | $\mathcal{L}(\mathsf{F})$ | $\mathcal{L}(\mathsf{G})$ |
|---|---|---|---|---|
| $\mathcal{L}(\mathsf{A})$ | | | | |
| $\mathcal{L}(\mathsf{E})$ | | | | |
| $\mathcal{L}(\mathsf{F})$ | | | | |
| $\mathcal{L}(\mathsf{G})$ | | | | |

**Solution:**

| | $\mathcal{L}(\mathsf{A})$ | $\mathcal{L}(\mathsf{E})$ | $\mathcal{L}(\mathsf{F})$ | $\mathcal{L}(\mathsf{G})$ |
|---|---|---|---|---|
| $\mathcal{L}(\mathsf{A})$ | ———— | ———— | ———— | $\epsilon$ |
| $\mathcal{L}(\mathsf{E})$ | ———— | ———— | ———— | $\epsilon$ |
| $\mathcal{L}(\mathsf{F})$ | d | d | ———— | $\epsilon$ |
| $\mathcal{L}(\mathsf{G})$ | ccdd | ccdd | ———— | ———— |

**Question 1b:** *Barbara is concerned about Giorgio's model. She has been told that grammars can lead to ambiguities, which can be the source of many problems.*

*To help Barbara:*

1. *Determine if the grammar is ambiguous or not. Write yes/no.*
2. *Provide a short argument in 1-3 lines, plus diagrams if you need them.*
3. *If the answer is yes, provide a grammar for the same language but without ambiguities. You do not need to prove correctness and non-ambiguity of your solution.*

**Solution:**

1. Yes, the grammar is ambiguous.
2. One can show this with two derivations or two parse trees for the string

   cdcdcd

3. Ambiguity arises from unspecified associativity in yuxtaposition in the body $SS$. One way to remedy this is to enforce left-associativity applying the method seen in class:

$$\begin{array}{lcl} S_0 & \rightarrow & S_0 S_1 \mid S_1 \\ S_1 & \rightarrow & \mathsf{cd} \mid \mathsf{c} S_0 \mathsf{d} \end{array}$$

**Question 1c:** *Barbara would like to know if the models provide at least as many donations as coffees are offered. Formally, she wants to check, for each model $M$, whether $\forall w \in \mathcal{L}(M).|w|_{\mathsf{d}} \geq |w|_{\mathsf{c}}$, where $|w|_s$ denotes the number of occurrences of symbol $s$ in $w$.*

*To help Barbara, determine which of the four models (A,E,F,G) have this property. For each model, write "yes" or "no" and explain your answer in 1-4 lines.*

**Solution:**

A: Yes. Any accepting path in A is a simple repetition of the unique loop through $s_0$, and said loop reads first d and then c, hence having the same number of d's and c's.

E: Yes, since $\mathcal{L}(A) = \mathcal{L}(E)$.

F: No. For example, $\mathsf{ccd} \in \mathcal{L}(F)$ but $|\mathsf{ccd}|_{\mathsf{d}} < |\mathsf{ccd}|_{\mathsf{c}}$.

G: Yes. We can use an induction proof/argument. The base case is production $S \rightarrow \mathsf{cd}$. The inductive cases are productions $S \rightarrow \mathsf{c}S\mathsf{d}$ and $S \rightarrow SS$. In both cases assuming each $S$ occurrence yields a word with more donations than coffees, leads to words with more donations than coffees.

**Question 1d:** *Elaborate on your answer in (c) regarding Giorgio's grammar* G *by providing a proof by induction to support your answer.*

**Solution:** The solution would be a detailed proof based on the sketch in the solution of (c) for G.

**Question 1e:** *Barbara is concerned about the memory limitations in the hardware she plans to use to build the coffee machine. She has learned in 02141 - Computer Science Modelling that regular languages just need finite memory but that this is not the case for non-regular languages.*

*Help Barbara as follows:*

- *Determine which of the four languages are regular: just write "Regular" or "Non-regular".*
- *Provide a short argument (1-4 lines) supporting your answer.*

**Solution:**

A: Regular. The language of any $\epsilon$-NFA is regular.

E: Regular. The language of any regular expression is regular.

F: Regular. Since, by the hint, any LTL formula can be expressed as a DFA and any such automata defines a regular language.

G: Non-regular. The language is very close to language of balanced parentheses. Indeed, replacing "(" by "d" and ")" by "c" one obtains the grammar for balanced parentheses. The pumping lemma proof can then be easily adjusted by renaming the symbols. Alternatively, just observe that regularity is closed under homomorphisms (hence under the renaming).

## Task 2

Alice has a pair of secret numbers: `a1` and `a2`. Bob has his own pair of secret numbers: `b1` and `b2`. Charlize needs to write a piece of code to notify Dave in a variable `diff` whether Alice's secrets coincide with Bob's secrets. Formally, Dave expects a program `C` that always reaches the final statement and that fulfils the following Hoare-Floyd triple:

```
{true}
C
{((a1 = b1 & a2 = b2) ==> diff = 0) & ((a1 != b1 | a2 != b2) ==> diff = 1)}
```

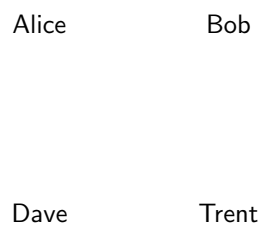
Charlize implements the below program `P`

```
diff := 0;
if
a1 != b1 -> diff := 1
[]
a1 = b1 ->
  if
  a2 != b2 -> diff := 1
  []
  a2 = b2 -> skip
  fi
fi
```

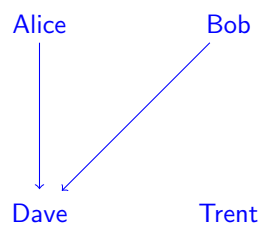
**Question 2a:** *Charlize is concerned about confidentiality issues. To begin with, she would like to do some information flow analysis (as seen in the course and as supported by* fm4fun *and* inspectify*).*

*She considers the security classification of the variables as follows:* $L(\texttt{a1}) =$ Alice, , $L(\texttt{a2}) =$ Alice , $L(\texttt{b1}) =$ Bob , $L(\texttt{b2}) =$ Bob , $L(\texttt{diff}) =$ Dave.

*Help Charlize by completing the diagram below. You have to add an arrow from security level* $L_1$ *to security level* $L_2$ *iff the information flow analysis determines that the program has a flow from some variable in* $L_1$ *to some variable in* $L_2$.




**Solution :**



**Question 2b:** *Charlize's program should ensure that only Dave can learn something about the secrets of Alice and Bob (besides each of them knowing their own secrets). In particular, Charlize wants to formalize the property "Alice cannot learn anything about Bob" using the concept of non-interference and she wants to verify the property using a corresponding valid Hoare-Floyd triple.*

*To help Charlize:*

- *Provide the Hoare-Floyd triple for the property "Alice cannot learn anything about Bob". Make sure that your answer adheres to rule* ***R3*** *(see front cover).*
- *Explain in 1-3 lines whether the property holds and why.*

**Solution :**

Following the technique seen in class, we can write the Hoare-Floyd triple below. Since it is verified in CHIP, we can conclude that the property holds.

```
{a1_1 = a1_2 & a2_1 = a2_2 & diff_1 = diff_2}
diff_1 := 0;
if
a1_1 != b1_1 -> diff_1 := 1
[]
a1_1 = b1_1 ->
  if
  a2_1 != b2_1 -> diff_1 := 1
  []
  a2_1 = b2_1 -> skip
  fi
fi
;
diff_2 := 0;
if
a1_2 != b1_2 -> diff_2 := 1
[]
a1_2 = b1_2 ->
  if
  a2_2 != b2_2 -> diff_2 := 1
  []
  a2_2 = b2_2 -> skip
  fi
fi
{a1_1 = a1_2 & a2_1 = a2_2 }
```

**Question 2c:** *Charlize is concerned with the possibility of anyone but Dave being able to learn whether Alice's and Bob's secrets coincide or differ just by measuring execution time of the program. She decides to introduce a variable* `time` *that estimates execution time by computing the number of operations executed in the program. Moreover, she decides to instrument the code according to the below program transformation function* instrument *which, given a GCL program, provides a new GCL program with time measurements.*

*To help Charlize:*

- *Provide the GCL program that results from applying the function* instrument *defined below to Charlize's program. Make sure that your answer adheres to rule* ***R3*** *(see front cover).*
- *Determine whether someone (e.g. Trent) can learn something about Alice's or Bob's secrets through variable* `time`*. If your answer is "no", provide a proof. If your answer is "yes", provide a counterexample.*

$$
\begin{array}{rcll}
\mathsf{instrument}(C) & = & \texttt{time := 0 ; } \mathsf{clock}(C) & \\
\mathsf{clock}(\texttt{skip}) & = & \texttt{skip} & \\
\mathsf{clock}(x\texttt{:=}a) & = & x \texttt{ := } a \texttt{ ; time := time + } 1 + \mathsf{clock}(a) & \\
\mathsf{clock}(C_1 \texttt{ ; } C_2) & = & \mathsf{clock}(C_1) \texttt{ ; } \mathsf{clock}(C_2) & \\
\mathsf{clock}(\texttt{if } GC \texttt{ fi}) & = & \texttt{if } \mathsf{clock}(GC) \texttt{ fi} & \\
\mathsf{clock}(\texttt{do } GC \texttt{ od}) & = & \texttt{do } \mathsf{clock}(GC) \texttt{ od} & \\
\mathsf{clock}(b \texttt{ -> } C) & = & b \texttt{ -> time := time} + \mathsf{clock}(b) \texttt{ ; } \mathsf{clock}(C) & \\
\mathsf{clock}(GC_1 \texttt{ [] } GC_2) & = & \mathsf{clock}(GC_1) \texttt{ [] } \mathsf{clock}(GC_2) & \\
\mathsf{clock}(k) & = & 0 & \text{if } k \text{ is a constant} \\
\mathsf{clock}(x) & = & 0 & \\
\mathsf{clock}(e_1 \bowtie e_2) & = & 1 + \mathsf{clock}(e_1) + \mathsf{clock}(e_2) & \text{if } \bowtie \text{ is a binary operator} \\
\mathsf{clock}(\bullet e) & = & 1 + \mathsf{clock}(e) & \text{if } \bullet \text{ is a unary operator} \\
\mathsf{clock}((e)) & = & (\mathsf{clock}(e)) &
\end{array}
$$

**Solution :**

Charlize's program after applying the transformation:

```
time := 0;
diff := 0;
time := time + 1;
if
a1 != b1 -> time := time + 1; diff := 1; time := time + 1
[]
a1 = b1 ->
  time := time + 1;
  if
  a2 != b2 -> time := time + 1; diff := 1; time := time + 1
  []
  a2 = b2 -> time := time + 1; skip
  fi
fi
```

To see that observers of `time` can learn something about Alice's and Bob's secrets it is sufficient to find two executions where the value of `time` is different at the end of the program.

For example, the below two initialisations

```
a1 = 0, a2 = 0, b1 = 0, b2 = 0, diff = 0, time = 0
a1 = 0, a2 = 1, b1 = 0, b2 = 0, diff = 0, time = 0
```

terminate with `time` being 3 and 4, respectively.

This can also be done using CHIP to disprove a non-interference property.

**Question 2d:** *Independently of the results of (c), Charlize is still concerned about time leaks. She has heard about* constant-time algorithms *and decides to implement a new program $C$ that*

- *is deterministic and always terminates without getting stuck;*
- *the following Hoare-Floyd triple is valid:*

  {`true`}
  $C$
  {`((a1 = b1 & a2 = b2) ==> diff = 0) & ((a1 != b1 | a2 != b2) ==> diff = 1)`}

- *execution* `time`*, computed according to the instrumentation function* `instrument` *provided in (c), is the same for all initial values of Alice's and Bob's secrets.*
- *does NOT use boolean operators (i.e.* `&`*,* `&&`*,* `|` *etc.). All other operators, e.g. relational operators (i.e.* `=`*,* `!=`*,* `>`*), arithmetic operators, etc. are allowed.*

*To help Charlize:*

1. *Provide the new program $C$. Your program must be instrumented with the time computation according to the instrumentation function* `instrument` *provided in (c). Make sure that your answer adheres to rule* ***R3*** *(see front cover).*
2. *Provide a valid Hoare-Floyd triple showing that the program has constant execution time. Make sure that your answer adheres to rule* ***R3*** *(see front cover).*

**Solution :**

1.

```
{true}
time := 0;
diff := 1;
time := time + 1;
if
a1 != b1 -> time := time + 1; diff := 0; time := time + 1
[]
a1 = b1 -> time := time + 1; diff := 1; time := time + 1
fi ;
if
a2 != b2 -> time := time + 1; diff := diff * 0; time := time + 2
[]
a2 = b2 -> time := time + 1; diff := diff * 1; time := time + 2
fi;
diff := diff - 1;  time := time + 2; diff := -1 * diff; time := time + 2
{((a1 = b1 & a2 = b2) ==> diff = 0) & ((a1 != b1 | a2 != b2) ==> diff = 1)}
```

2.

```
{true}
time := 0;
diff := 1;
time := time + 1;
if
a1 != b1 -> time := time + 1; diff := 0; time := time + 1
[]
a1 = b1 -> time := time + 1; diff := 1; time := time + 1
fi ;
if
a2 != b2 -> time := time + 1; diff := diff * 0; time := time + 2
[]
a2 = b2 -> time := time + 1; diff := diff * 1; time := time + 2
fi;
diff := diff - 1;  time := time + 2; diff := -1 * diff; time := time + 2
{time = 10}
```

# Task 3

In this task, we extend the guarded command language used throughout the course (see [FM, Definition 2.3]) by a new command of the form

$$x, y \ := \ \mathsf{split}(a_1, a_2)$$

Informally, the above command takes two arithmetic expressions $a_1$ and $a_2$ as an input, evaluates both in the current memory, and assigns the smaller of the two resulting values to variable $x$ and the larger of those two values to variable $y$, respectively.

**Question 3a:** *Let $\sigma$ be the program memory given by the following table*

| $x$ | $\sigma(x)$ |
|---|---|
| $u$ | *17* |
| $v$ | *23* |
| $w$ | *42* |

*Give a program memory $\sigma'$ such that $\sigma' = \mathcal{S}[\![u, v := \ \mathit{split}(u+v, w-v)]\!](\sigma)$, where you should assume that $\mathcal{S}[\![u, v := \ \mathit{split}(u+v, w-v)]\!]$ adequately captures the behavior informally described above.*

***Briefly (1-2 sentences) explain your answer.***

**Solution :**

$\mathcal{A}[\![u+v]\!](\sigma) = 40$
$\mathcal{A}[\![w-v]\!](\sigma) = 19$
So 19 is assigned to $u$ and 40 is assigned to $v$. The resulting memory is:

| $x$ | $\sigma(x)$ |
|---|---|
| $u$ | 19 |
| $v$ | 40 |
| $w$ | 42 |

**Question 3b:** *Formally define the semantics of $x, y \ := \ \mathit{split}(a_1, a_2)$ such that it captures the behavior informally described above for any given memory $\sigma$.* ***Briefly (1-3 sentences) explain your answer.***

$$\mathcal{S}[\![x, y \ := \ \mathit{split}(a_1, a_2)]\!](\sigma) \quad =$$

**Solution :**

$$\mathcal{S}[\![x, y \ := \ \mathsf{split}(a_1, a_2)]\!](\sigma) \quad =$$
$$\begin{cases} \sigma[x \mapsto \min(z_1, z_2)][y \mapsto \max(z_1, z_2)] & \text{if } z_i = \mathcal{A}[\![z_i]\!](\sigma) \in \mathbb{Z}, i \in \{1, 2\} \\ \mathsf{undefined} & \text{otherwise} \end{cases}$$

**Question 3c:** *Provide a postcondition $Q$ such that the Floyd-Hoare triple*

$$\{u = N \wedge v = M\}\ C\ \{Q\}$$

*is valid if and only if the command $C$ "behaves like" the command $x, y := \mathsf{split}(u, v)$. Here, $N, M$ are variables that are never accessed by $C$. More precisely, for every complete execution sequence of $C$ with initial memory $\sigma$ and final memory $\sigma'$, it should hold that $\sigma' \models Q$ if and only if there is a memory $\sigma''$ such that*

1. $\mathcal{S}[\![x, y := \mathsf{split}(u, v)]\!](\sigma) = \sigma''$ *and*
2. $\sigma''(x) = \sigma'(x)$ *and*
3. $\sigma''(y) = \sigma'(y)$.

*You do* not *have to provide a command $C$ for this task; the postcondition $Q$ suffices.* ***Briefly (1-3 sentences) explain your answer.***

**Solution :** A possible postcondition $Q$ is given by:

$$\begin{array}{ll} \wedge & (N < M \rightarrow x = N \wedge y = M) \\ \wedge & (\neg(N < M) \rightarrow x = M \wedge y = N) \end{array}$$

Notice that nothing has to be done for requirement 2 and 3; the requirement simply makes sure that we do not have to talk about other variables.

**Question 3d:** *Show that the command $x, y := \mathsf{split}(2 * x + 1, x + y)$ can be simulated by a program written in the original guarded command language used in the course (see [FM, Definition 2.3]).*

*To this end, provide a* ***fully annotated*** *command of the form*

$$\{x = N \wedge y = M\}\ AC\ \{Q\}\ ,$$

*such that*

1. *$Q$ corresponds to the postcondition developed in part (c) if $u$ stores the value of $2 * x + 1$ and $v$ stores the value of $x + y$,*
2. *the variables $N$ and $M$ do not appear in the command described by $AC$ (but they may appear in annotations),*
3. *the command described by $AC$ always terminates (i.e., there are no infinitely long execution sequences),*
4. *the command described by $AC$ never gets stuck, and*
5. *the fully annotated command must verify with* chip*. Make sure that your answer adheres to rule* ***R3*** *(see front cover).*

*In addition to providing the fully annotated command,* ***briefly (1-2 sentences) explain*** *why your command always terminates and* ***prove*** *that your command never gets stuck.*

**Solution :** The main idea is to first assign the values obtained from evaluating $2 * x + 1$ and $x + y$ to fresh variables $u, v$. After that, we can essentially re-use the previously developed postcondition. The command itself is simply an implementation of the informally described split function. For example, here is one possible fully annotated guarded command:

```
{ x = N & y = M }
u := 2 * x + 1;
{ x = N & y = M & u = 2 * N + 1 }
v := x + y;
{ x = N & y = M & u = 2 * N + 1 & v = N + M }
if
   u < v -> x := u; y := v
[] !(u < v) -> x := v; y := u
fi
{ (u < v ==> x = u & y = v) & (!(u < v) ==> x = v & y = u) }
```

The command clearly terminates because it is loop-free. It also never gets stuck. To see this, it suffices to check the proposition on evolving systems from the [FM] book. We first notice that all expressions are always defined and thus no edge labeled with an assignment can get stuck. There are only two edges labeled with Boolean expressions and both have the same source node. Those edges are labeled $u < v$ and $!(u < v)$. Hence, for every $\sigma$, either $\mathcal{S}[\![u < v]\!](\sigma)$ or $\mathcal{S}[\![!(u < v)]\!](\sigma)$ is defined, i.e. the program graph underlying the above command constitutes an evolving system and can thus never get stuck.

# Task 4

Consider the following command $C$:

```
i := 0;
j := 0;
s := 0;
t := 0;
do
  i < n ->
    s := s + 2*i;
    i := i + 1
[] j < m ->
    t := t + 4*j*j*j;
    j := j + 1
od;
do
  m != N | n != N ->
    s := s + 1;
    s := s - 1
od;
r := t - s
```

*Hint:* The questions in this task are independent of each other. Make sure that you read all of them, even if you decide to omit answering a question.

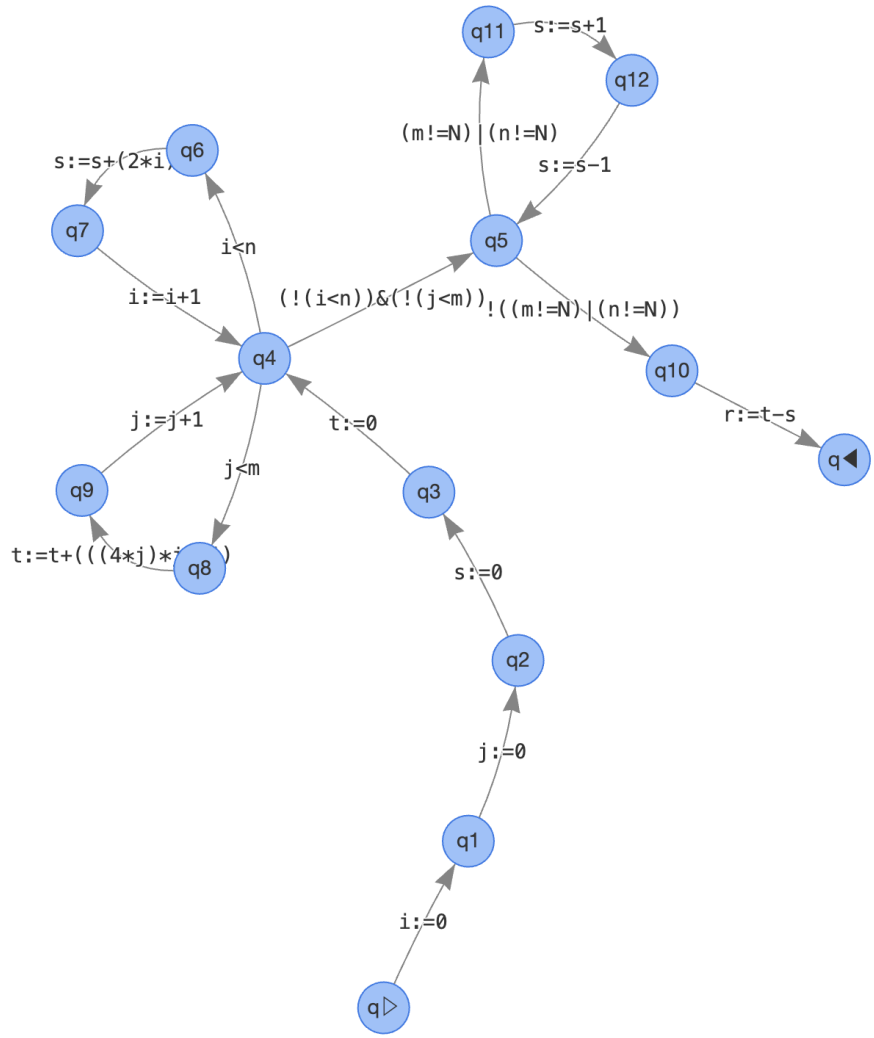
**Question 4a:** *Give a Floyd-Hoare triple stating that the loop body of the second loop of $C$, i.e. the program* `s := s + 1; s := s - 1` *does not change program variable* `s`.

**Solution :**

$$\{s = S\}\texttt{s := s + 1; s := s - 1}\{s = S\}$$

**Question 4b:** *Provide the program graph of $C$.*

**Solution :**




**Question 4c:** *For each of the following statements,* ***explain (in 1-3 sentences for each statement)*** *whether it is* `true`*,* `false`*, or* `unknown` *(i.e. no conclusive answer is possible). For each statement, your answers must be* <u>*based solely*</u> *on the results of a detection of signs analysis as defined in [FM]. You may assume that formalmethods.dk correctly implements the analysis in [FM].*

*(i) After the initial assignment to $t$ in line 4, it always holds that $t >= 0$.*

*(ii) $s > 0$ holds right after the first loop terminates.*

*(iii) $s >= 0$ holds right after the second loop terminates.*

*(iv) If $N$ is initially positive, then there is at least one complete execution sequence such that $r > 0$ holds in the final node.*

*(v) There is no complete execution sequence if initially $N \leq 0$ holds.*

**Solution :**

(i) True. In every node after said assignment, the analysis assigns only 0 or + to $t$.

(ii) Unknown. According to the analysis, 0 is also possible in $q_4$ but we do not know whether such a result is realizable.

(iii) Unknown. According to the analysis, $-$ is also possible in $q_{10}$ but we do not know whether such a result is realizable.

(iv) Unknown. The analysis shows some abstract memories in which the finlal node is reached but we do not know whether there is indeed a complete execution sequence.

(v) Unknown. For $N = 0, i = 0, j = 0, m = 0, n = 0, r = 0, s = 0, t = 0$, the analysis finds a final memory but that memory might not be realizable.

**Question 4d:** *Show that the following Floyd-Hoare triple is valid for the program $C$ by providing a suitable fully annotated command:*

$$\{n >= 0 \wedge m >= 0\}\ C\ \{r = N * (N - 1) * (N * N - N - 1)\}$$

*Make sure that your answer adheres to rule **R3** (see front cover).*

***Solution :***

Chip

```
{ n >= 0 & m >= 0 }
i := 0;
{ n >= 0 & m >= 0 & i = 0 }
j := 0;
{ n >= 0 & m >= 0 & i = 0 & j = 0 }
s := 0;
{ n >= 0 & m >= 0 & i = 0 & j = 0 & s = 0 }
t := 0;
{ i <= n & s = i * (i-1) & j <= m & t = j*j * (j-1) * (j-1) }
do[i <= n & s = i * (i-1) & j <= m & t = j*j * (j-1) * (j-1)]
  i < n ->
    { i < n & i <= n & s = i * (i-1) & j <= m & t = j*j * (j-1) * (j-1)}
    { i+1 <= n & s + 2*i = (i+1) * (i) & j <= m & t = j*j * (j-1) * (j-1) }
    s := s + 2*i;
    { i+1 <= n & s = (i+1) * (i) & j <= m & t = j*j * (j-1) * (j-1) }
    i := i + 1
    { i <= n & s = i * (i-1) & j <= m & t = j*j * (j-1) * (j-1) }
[] j < m ->
    { j < m & i <= n & s = i * (i-1) & j <= m & t = j*j * (j-1) * (j-1) }
    { j < m & i <= n & s = i * (i-1) & j+1 <= m & t + 4*j*j*j = (j+1)*(j+1) * (j) * (j) }
    t := t + 4*j*j*j;
    { i <= n & s = i * (i-1) & j+1 <= m & t = (j+1)*(j+1) * (j) * (j) }
    j := j + 1
    { i <= n & s = i * (i-1) & j <= m & t = j*j * (j-1) * (j-1) }
od;
{ i = n & j = m & t = m*m * (m-1) * (m-1) & s = n * (n-1) }
do[t = m*m * (m-1) * (m-1) & s = n * (n-1)]
  m != N | n != N ->
    { t = m*m * (m-1) * (m-1) & s = n * (n-1) }
    s := s + 1;
    { t = m*m * (m-1) * (m-1) & s-1 = n * (n-1) }
    s := s - 1
    { t = m*m * (m-1) * (m-1) & s = n * (n-1) }
od;
{ m = N & n = N & t = m*m * (m-1) * (m-1) & s = n * (n-1) }
r := t - s
{ r = N * (N-1) * (N*N - N - 1)}
```

*The latex sources contain a copyable version.*

**Question 4e:** *Show that the command $C$ does* not *always terminate by providing an execution sequence demonstrating nontermination. Briefly explain why your execution sequence proves nontermination.*

***Solution :*** *The following execution demonstrates nontermination. It suffices to observe that the second loop is deterministic and that the loop guard of the second loop reaches exactly the same state twice.*

| | Action | Node | N | i | j | m | n | r | s | t |
|---|---|---|---|---|---|---|---|---|---|---|
| | | q▷ | 0 | 0 | 0 | 2 | 1 | 0 | 0 | 0 |
| | i:=0 | q1 | 0 | 0 | 0 | 2 | 1 | 0 | 0 | 0 |
| | j:=0 | q2 | 0 | 0 | 0 | 2 | 1 | 0 | 0 | 0 |
| | s:=0 | q3 | 0 | 0 | 0 | 2 | 1 | 0 | 0 | 0 |
| | t:=0 | q4 | 0 | 0 | 0 | 2 | 1 | 0 | 0 | 0 |
| | j<m | q8 | 0 | 0 | 0 | 2 | 1 | 0 | 0 | 0 |
| | t:=t+(((4*j)*j)*j) | q9 | 0 | 0 | 0 | 2 | 1 | 0 | 0 | 0 |
| | j:=j+1 | q4 | 0 | 0 | 1 | 2 | 1 | 0 | 0 | 0 |
| | i<n | q6 | 0 | 0 | 1 | 2 | 1 | 0 | 0 | 0 |
| | s:=s+(2*i) | q7 | 0 | 0 | 1 | 2 | 1 | 0 | 0 | 0 |
| | i:=i+1 | q4 | 0 | 1 | 1 | 2 | 1 | 0 | 0 | 0 |
| | j<m | q8 | 0 | 1 | 1 | 2 | 1 | 0 | 0 | 0 |
| | t:=t+(((4*j)*j)*j) | q9 | 0 | 1 | 1 | 2 | 1 | 0 | 0 | 4 |
| | j:=j+1 | q4 | 0 | 1 | 2 | 2 | 1 | 0 | 0 | 4 |
| | (!(i<n))&(!(j<m)) | q5 | 0 | 1 | 2 | 2 | 1 | 0 | 0 | 4 |
| | (m!=N)\|(n!=N) | q11 | 0 | 1 | 2 | 2 | 1 | 0 | 0 | 4 |
| | s:=s+1 | q12 | 0 | 1 | 2 | 2 | 1 | 0 | 1 | 4 |
| | s:=s-1 | q5 | 0 | 1 | 2 | 2 | 1 | 0 | 0 | 4 |
| | (m!=N)\|(n!=N) | q11 | 0 | 1 | 2 | 2 | 1 | 0 | 0 | 4 |

*Alternative solution: use MOKA to model check the property "eventually terminated" to get a counterexample. See the LaTeX for the code.*

**Question 4f:** *Consider the following program analysis problem: Given a program graph, assign to every node $q$ the set $A(q)$ of those memories $\sigma$ such that there exists an execution sequence of the form $\langle q;\sigma\rangle \Rightarrow \ldots \Rightarrow \langle q_{\blacktriangleleft};\sigma'\rangle$. Show (without using Rice's theorem) that there is no algorithm that takes a program graph $G$ as an input and computes $A(q)$ for every node $q$ of $G$.*

*Hint: You may assume that there is an algorithm for checking whether a memory is contained in $A(q)$.*

**Solution :**

Assume there is an algorithm that computes the above analysis information. We reduce the described problem to the Halting problem, which is well-known (from the course) to be undecidable. Given a command $C$ and an initial memory $\sigma$, we can then decide the halting problem by first running the analysis on the (deterministic) program graph of $C$ and then checking whether $\sigma \in A(q_{\triangleright})$. This is true iff $C$ terminates on $\sigma$.